E7 — Mise en situation professionnelle

Certification Ingénieur en Infrastructure Massive de Données

Candidat : Mezine Elias

Bloc de compétences : Bloc 4 — Encadrer la collecte massive et la mise à disposition des données issues de l'activité de l'organisation avec un data lake

Livrable : E7 — Encadrer la collecte massive et la mise à disposition des données avec un Data Lake

---

Sommaire

Conception de l'architecture du Data Lake **3**
1. Contexte et cadre d'exploitation . . . 3
   1.1 Présentation du projet . . . 3
   1.2 Cadre technique de l'organisation . . . 3
2. Analyse des contraintes (3V) . . . 3
   2.1 Volume . . . 3
   2.2 Velocité . . . 3
   2.3 Variété . . . 3
3. Architecture technique . . . 4
   3.1 Schéma d'architecture . . . 4
   3.2 Architecture en couches du Data Lake . . . 5
4. Comparatif des outils de catalogue de données . . . 6
   4.1 Solutions évaluées . . . 6
   4.2 Analyse comparative . . . 6
   4.3 Matrice de décision . . . 6
5. Solution retenue : Azure Purview . . . 7
   5.1 Justification du choix . . . 7
   5.2 Validation financière . . . 7

Intégration des composants d'infrastructure du Data Lake **8**
1. Prérequis . . . 8
2. Procédure d'installation . . . 8
   2.1 Authentification . . . 8
   2.2 Déploiement . . . 8
3. Système de stockage — Data Lake . . . 8
4. Outils batch — Data Factory & Databricks . . . 8
   4.1 Azure Data Factory . . . 8
   4.2 Azure Databricks . . . 9
5. Catalogue de données — Azure Purview . . . 10

5.1 Enregistrement des sources . . . 10
5.2 Scans et résultats . . . 11

Gestion du catalogue des données et conformité RGPD **12**
1. Sources de données et méthodes d'alimentation . . . 12
2. Intégration des métadonnées dans le catalogue . . . 12
3. Données personnelles et RGPD . . . 14
3.1 Registre des traitements . . . 14
3.2 Procédure de tri des données personnelles . . . 14
3.3 Données personnelles identifiées . . . 15
3.4 Procédures de conformité RGPD . . . 16
4. Procédures de suppression . . . 16
4.1 Politique de rétention du Data Lake (Lifecycle Management Policy) . . . 16
4.2 Purge des données personnelles en base SQL . . . 17
5. Monitorage . . . 17
5.1 Alertes configurées . . . 17
5.2 Collecte des logs . . . 18

Implémentation des règles de gouvernance des données **20**
1. Modèle de sécurité . . . 20
1.1 Principes appliqués . . . 20
2. Accès par groupes dans Azure Purview . . . 20
2.1 Création des groupes dans Microsoft Entra ID . . . 20
2.2 Attribution des rôles Purview aux groupes . . . 20
2.3 Accès limités aux ressources nécessaires . . . 21
3. Mécanismes d'authentification et droits des services . . . 21
3.1 Comparatif des mécanismes d'authentification . . . 21
3.2 Choix et justifications . . . 21
3.3 Matrice des droits par ressource . . . 22
3.4 Contribution à la conformité RGPD . . . 23
4. Procédures de gestion des accès . . . 23

Synthèse **24**

Conception de l'architecture du Data Lake

1. Contexte et cadre d'exploitation

1.1 Présentation du projet

Ce projet vise à mettre en place une plateforme de données Azure permettant : - La collecte de données massives depuis diverses sources - Le stockage structuré dans un Data Lake - Le traitement et la transformation des données - La gouvernance et le catalogage des actifs de données - La mise à disposition pour l'analyse et le reporting

1.2 Cadre technique de l'organisation

Élément	Choix
Cloud Provider	Microsoft Azure
Infrastructure as Code	Terraform
Région principale	France Central
Région secondaire	North Europe (Purview)
Modèle de sécurité	Managed Identities + RBAC

2. Analyse des contraintes (3V)

2.1 Volume

Contrainte	Analyse	Impact architectural
Taille des données	Plusieurs Go à To de données	Nécessite un stockage scalable et économique
Croissance	Volume croissant avec l'ajout de nouvelles sources	Besoin de tiers de stockage Hot/Cool/Archive
Historisation	Conservation longue durée	Architecture en couches médaillon

Solution retenue : Azure Data Lake Storage Gen2 avec Hierarchical Namespace pour optimiser les performances sur les gros volumes.

2.2 Velocité

Contrainte	Analyse	Impact architectural
Fréquence d'ingestion	Batch journalier/hebdomadaire	Azure Data Factory pour l'orchestration
Latence acceptable	Plusieurs minutes	Pas de besoin temps réel immédiat
Fenêtre de traitement	Nuit/heures creuses	Possibilité de traitements lourds

Solution retenue : Architecture batch avec Azure Data Factory.

2.3 Variété

Contrainte	Analyse	Impact architectural
Types de sources	SQL, fichiers, APIs	Connecteurs multiples nécessaires
Formats de données	CSV, JSON, Parquet	Stockage agnostique au format
Structuration	Structuré et semi-structuré	Schema-on-read avec Data Lake

Solution retenue : Data Lake avec containers dédiés par niveau de traitement, supportant tous les formats.

3. Architecture technique

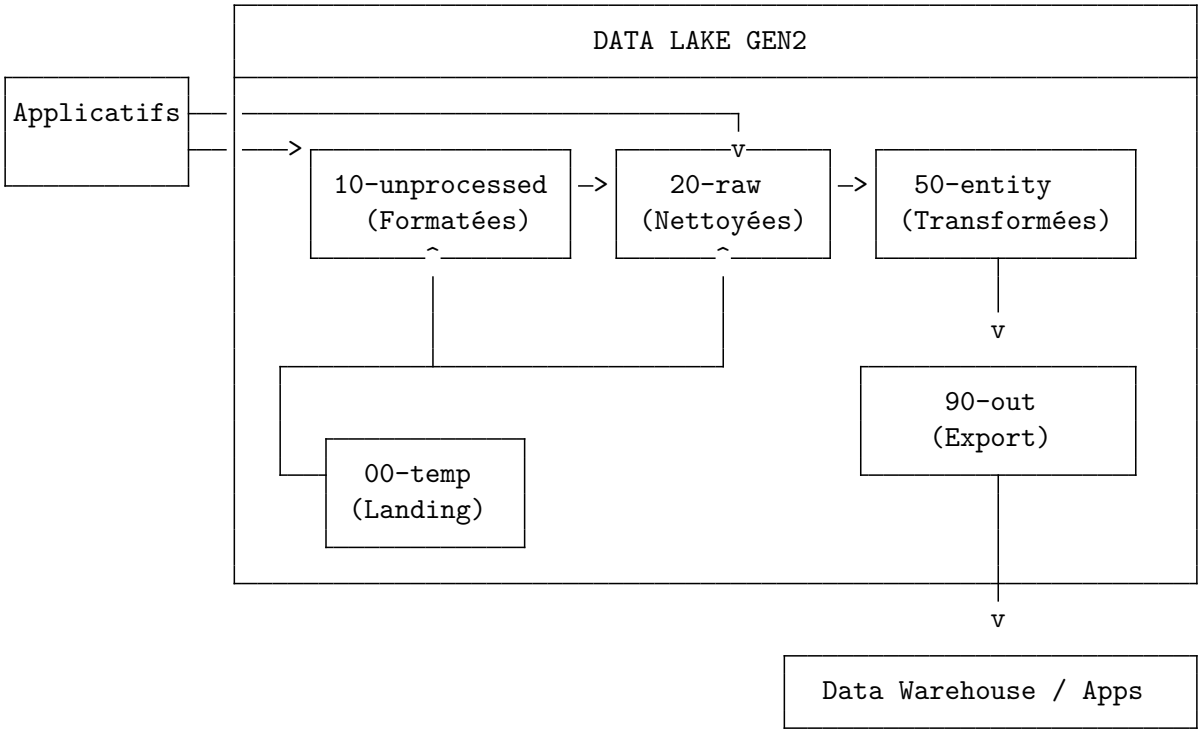
3.1 Schéma d'architecture

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                   Resource Group: mezinenhood
                                       (France Central)
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│
│   ┌──────────────────────┐          ┌──────────────────────┐          ┌──────────────────────┐
│   │   Data Lake          │          │   Azure SQL DB       │          │    Key Vault         │
│   │  (ADLS Gen2)         │          │                      │          │                      │
│   │                      │          │  mezinebasenhood     │          │  Secrets:            │
│   │ Containers:          │          │                      │          │  - sql-admin-pwd     │
│   │ - 00-temp            │          └──────────^───────────┘          │  - app-secret        │
│   │ - 10-unprocessed     │                     │                      └──────────^───────────┘
│   │ - 20-raw             │                     │                                 │
│   │ - 50-entity          │                     │                                 │
│   │ - 90-out             │                     │                                 │
│   │ - tfstate            │                     │                                 │
│   └──────────^───────────┘                     │                                 │
│              │                                 │                                 │
│              │                                 │                                 │
│              │                                 
```

Nom ↑		Type
ADF-Pipeline-Failure-Alert	···	Règle d'alerte de métrique
Global-Email-Alert	···	Groupe d'actions
mezine-law-nhood	···	Espace de travail Log Analytics
mezinebasenhood (mezineserveur		Base de données SQL
mezinedatabricksworkspace	···	Service Azure Databricks
mezinefactorynhood2	···	Data Factory (V2)
mezinekeyvaultnhood2	···	Coffre de clés
mezinepurviewnhood2	···	Compte Microsoft Purview
mezineserveursqlnhood2	···	Serveur SQL
mezinestorageaccnhood2	···	Compte de stockage
SQL-DTU-90	···	Règle d'alerte de métrique
Storage-Capacity-80	···	Règle d'alerte de métrique

Figure 1: Vue d'ensemble du Resource Group Azure

3.2 Architecture en couches du Data Lake



- **Landing** (`00-temp`) : zone d'entrée éphémère pour les fichiers bruts entrants, acheminés ensuite vers les couches supérieures.
- **Brutes fiables** (`10-unprocessed`) : données converties dans un format structuré exploitable. En pratique, elles transitent souvent directement vers la couche 20.
- **Nettoyées** (`20-raw`) : données ayant subi un nettoyage technique (standardisation, validation) pour garantir leur qualité.
- **Transformées** (`50-entity`) : couche métier implémentant le modèle en étoile. Les données y sont agrégées et prêtes pour l'analyse.
- **Export** (`90-out`) : données finales exposées aux applications métier et au Data Warehouse pour le reporting.

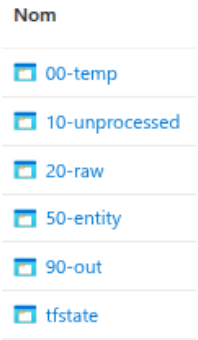


Figure 2: Containers du Data Lake

4. Comparatif des outils de catalogue de données

4.1 Solutions évaluées

Critère	Azure Purview	Apache Atlas	Alation	Collibra
Intégration Azure	Native	Manuelle	Connecteurs	Connecteurs
Coût	Pay-as-you-go	Open Source	Licence	Licence
Déploiement	PaaS managé	Self-hosted	SaaS/On-prem	SaaS/On-prem
Scan automatique	Oui	Limité	Oui	Oui
Classification données	IA intégrée	Manuel	IA	IA
RBAC natif Azure	Oui	Non	Partiel	Partiel
Managed Identity	Oui	Non	Non	Non

4.2 Analyse comparative

Azure Purview **Avantages** : - Intégration native avec l'écosystème Azure (Data Factory, Synapse, Data Lake) - Classification automatique des données sensibles (RGPD) - Authentification via Managed Identity (pas de credentials à gérer) - Modèle de coût pay-as-you-go adapté au projet

Inconvénients : - Disponibilité régionale limitée (nécessite North Europe) - Vendor lock-in Microsoft

Apache Atlas **Avantages** : - Open Source, pas de coût de licence - Indépendant du cloud provider

Inconvénients : - Nécessite infrastructure dédiée (VMs, maintenance) - Intégration Azure manuelle - Pas de classification automatique

Alation / Collibra **Avantages** : - Solutions matures, fonctionnalités avancées - Multi-cloud

Inconvénients : - Coût de licence élevé - Complexité de déploiement - Pas d'intégration native Managed Identity

4.3 Matrice de décision

Critère	Poids	Purview	Atlas	Alation	Collibra
Intégration Azure	35%	5	2	3	3
Coût	30%	4	5	2	2
Facilité déploiement	20%	5	2	3	3
Sécurité (RBAC/MI)	15%	5	2	3	4
Score pondéré	100%	**4.70**	2.90	2.70	2.85

5. Solution retenue : Azure Purview

5.1 Justification du choix

Azure Purview est sélectionné pour les raisons suivantes :

1. **Cohérence avec le cadre technique** : Intégration native avec Azure Data Factory, Data Lake et SQL Database déjà déployés
2. **Sécurité** : Utilisation des Managed Identities, pas de secrets à gérer pour les connexions
3. **Gouvernance RGPD** : Classification automatique des données personnelles
4. **Coût maîtrisé** : Modèle pay-as-you-go sans licence initiale

5.2 Validation financière

Une estimation des coûts a été réalisée avec **Infracost** pour valider la viabilité économique de l'architecture retenue :

Name	Monthly Qty	Unit	Monthly Cost
azurerm_mssql_database.mezinesqldatabase			
↳ Compute (BASIC)	730	hours	$6.11
↳ Long-term retention (RA-GRS)	Cost depends on usage: $0.0625 per GB		
↳ PITR backup storage (RA-GRS)	Cost depends on usage: $0.25 per GB		
azurerm_data_factory.adf Tags: CreatedBy=Elias Mezine, ManagedBy=Terraform			
↳ Read/Write operations	Cost depends on usage: $0.50 per 50k entities		
↳ Monitoring operations	Cost depends on usage: $0.25 per 50k entities		
azurerm_databricks_workspace.this Tags: CreatedBy=Elias Mezine, ManagedBy=Terraform			
↳ All-purpose compute DBUs	Cost depends on usage: $0.40 per DBU-hours		
↳ Jobs compute DBUs	Cost depends on usage: $0.15 per DBU-hours		
↳ Jobs light compute DBUs	Cost depends on usage: $0.07 per DBU-hours		
azurerm_log_analytics_workspace.law Tags: CreatedBy=Elias Mezine, ManagedBy=Terraform			
↳ Log data ingestion	Cost depends on usage: $2.76 per GB		
↳ Log data export	Cost depends on usage: $0.13 per GB		
↳ Basic log data ingestion	Cost depends on usage: $0.63 per GB		
↳ Basic log search queries	Cost depends on usage: $0.00625 per GB searched		
↳ Archive data	Cost depends on usage: $0.025 per GB		
↳ Archive data restored	Cost depends on usage: $0.13 per GB		
↳ Archive data searched	Cost depends on usage: $0.00625 per GB		
azurerm_monitor_action_group.email_alert Tags: CreatedBy=Elias Mezine, ManagedBy=Terraform			
↳ Email notifications (1)	Cost depends on usage: $0.00002 per emails		
azurerm_storage_account.datalake Tags: CreatedBy=Elias Mezine, ManagedBy=Terraform			
↳ Capacity	Cost depends on usage: $0.0192 per GB		
↳ Write operations	Cost depends on usage: $0.059 per 10k operations		
↳ List and create container operations	Cost depends on usage: $0.059 per 10k operations		
↳ Read operations	Cost depends on usage: $0.0047 per 10k operations		
↳ All other operations	Cost depends on usage: $0.0047 per 10k operations		
↳ Blob index	Cost depends on usage: $0.0376 per 10k tags		
Project total			**$6.11**

Overall total	**$6.11**

33 cloud resources were detected:
· 10 were estimated
· 22 were free
· 1 is not supported yet, see https://infracost.io/requested-resources:
· 1 x azurerm_purview_account

Figure 3: Rapport Infracost

Intégration des composants d'infrastructure du Data Lake

L'ensemble de l'infrastructure est déployée et automatisée avec Terraform (Infrastructure as Code), garantissant un déploiement reproductible.

1. Prérequis

L'installation nécessite les outils suivants sur la machine de déploiement :

Outil	Version	Vérification
Terraform	>= 1.0	`terraform version`
Azure CLI	>= 2.50	`az version`
sqlcmd	Installé	`sqlcmd -?`

Le compte Azure doit disposer des rôles Contributor et User Access Administrator sur la subscription.

2. Procédure d'installation

2.1 Authentification

```
az login
az account set --subscription "<SUBSCRIPTION_ID>"
```

2.2 Déploiement

> **Prérequis** : Le conteneur `tfstate` du Storage Account doit être créé manuellement avant la première exécution de `terraform init`.

```
terraform init
terraform plan -var-file="terraform.tfvars"
terraform apply -var-file="terraform.tfvars"
```

Le déploiement crée les ressources dans l'ordre suivant : Resource Group → Storage Account → Key Vault → SQL Database → Data Factory → Databricks → Purview.

3. Système de stockage — Data Lake

Le Data Lake est déployé avec six containers (`00-temp`, `10-unprocessed`, `20-raw`, `50-entity`, `90-out`, `tfstate`).

4. Outils batch — Data Factory & Databricks

La fréquence d'ingestion est journalière ou hebdomadaire, avec une latence acceptable de plusieurs minutes. Un traitement en temps réel n'est pas requis pour ce projet. L'architecture batch avec Azure Data Factory répond pleinement à ces contraintes de vitesse.

4.1 Azure Data Factory

Trois Linked Services connectent Data Factory aux sources de données :

Linked Service	Cible	Authentification
`mezine-ls-dtl`	Data Lake	Managed Identity
`ls-keyvault`	Key Vault	Managed Identity
`ls-azuresql`	SQL Database	Key Vault (password)

Pour chaque Linked Service, le test de connexion retourne "Connection successful".

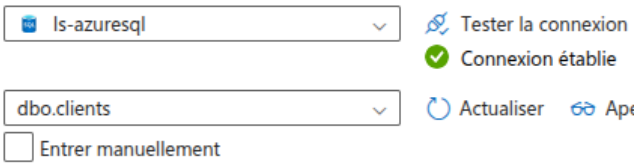


Figure 4: Test de connexion réussi — Linked Service ls-azuresql

Un pipeline de copie de données a été exécuté entre les différentes couches du Data Lake pour valider le fonctionnement de l'orchestration.

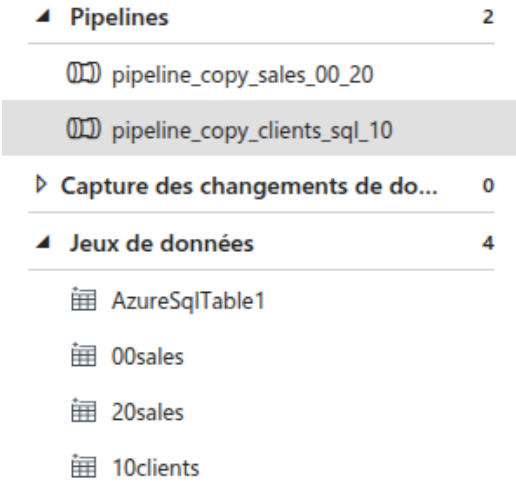


Figure 5: Pipelines et datasets dans ADF Studio

4.2 Azure Databricks

Le workspace Databricks est configuré avec : - Un cluster Spark `mezineclusternhood` - Un Secret Scope `mezinescopenhood` lié au Key Vault - Deux notebooks dans `/Shared/data_acces/`

Accès au Data Lake : le notebook `mount_adls` monte les containers du Data Lake via le connecteur ABFS.

```
print(dbutils.fs.ls(f"/mnt/{container_names[0]}"))
print(dbutils.fs.ls(f"abfss://{container_names[0]}@{storage_acc_name}.dfs.core.windows.net/"))

[FileInfo(path='dbfs:/mnt/00-temp/sales.csv', name='sales.csv', size=16601, modificationTime=1768848951000)]
[FileInfo(path='abfss://00-temp@mezinestorageaccnhood.dfs.core.windows.net/sales.csv', name='sales.csv', size=16601, modificationTime=1768848951000)]
```

Figure 6: Notebook mount_adls exécuté avec succès

Accès SQL : le notebook `dbconnexion` lit la table `clients` de la base SQL Azure.

```
id: integer
name: string
email: string
```

Table +

	id	name	email
1	929	Client_2BAFA	client_F064F0FF@test.com
2	1109	Client_19A74	client_00E82DD1@test.com
3	1324	Client_6ED77	client_DE4463C5@test.com
4	1961	Client_4CF93	client_A2A8F6A5@test.com
5	5371	Client_D25FF	client_528AC82F@test.com
6	5848	Client_13AD8	client_6B4AC085@test.com
7	6363	Client_B27B7	client_FFF9E8B6@test.com
8	7841	Client_937DC	client_DD02B556@test.com
9	8099	Client_AD703	client_2CD8FA86@test.com
10	9186	Client_604C1	client_EC65ED20@test.c...

Figure 7: DataFrame de la table clients via connexion JDBC

5. Catalogue de données — Azure Purview

5.1 Enregistrement des sources

Deux sources sont enregistrées dans Purview :

Source	Type	Authentification
`mezinestorageaccnhood2`	ADLS Gen2	Managed Identity
`mezinebasenhood`	Azure SQL	Key Vault (credentials SQL)

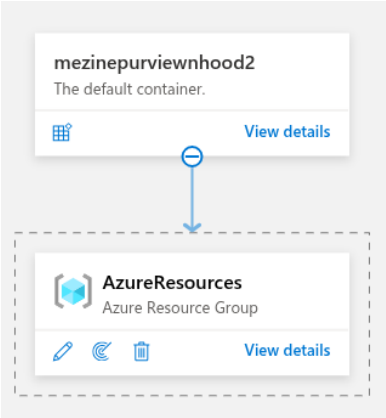


Figure 8: Collections et sources dans Purview Data Map

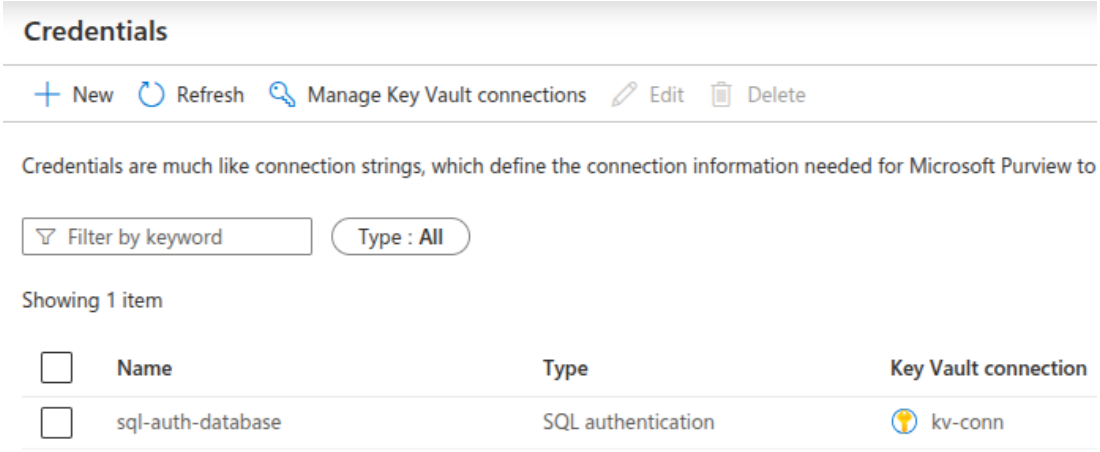


Figure 9: Credentials Purview — authentification SQL via Key Vault

5.2 Scans et résultats

Un scan hebdomadaire est configuré pour chaque source. Après exécution, les assets suivants apparaissent dans le catalogue : - 6 containers du Data Lake - Table `clients` de la base SQL

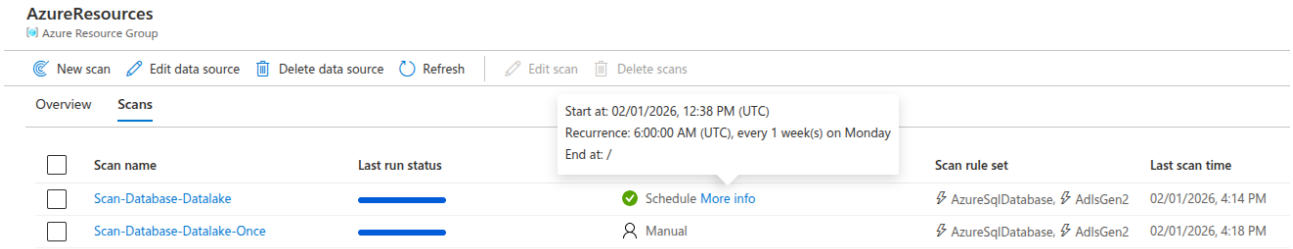


Figure 10: Configuration des scans Purview — récurrence hebdomadaire

Gestion du catalogue des données et conformité RGPD

1. Sources de données et méthodes d'alimentation

Source	Type	Données	Méthode d'alimentation
SQL Database `mezinebasenhood`	Base relationnelle	Table `clients` (id, name, email)	notebook Spark Databricks
Fichier `sales.csv`	CSV	Données de ventes	Azure Data Factory (batch)

La table `clients` est créée et alimentée via un notebook Spark dans Azure Databricks, qui permet la création du schéma et l'alimentation via JDBC/Spark, adapté à un traitement programmatique de données relationnelles.

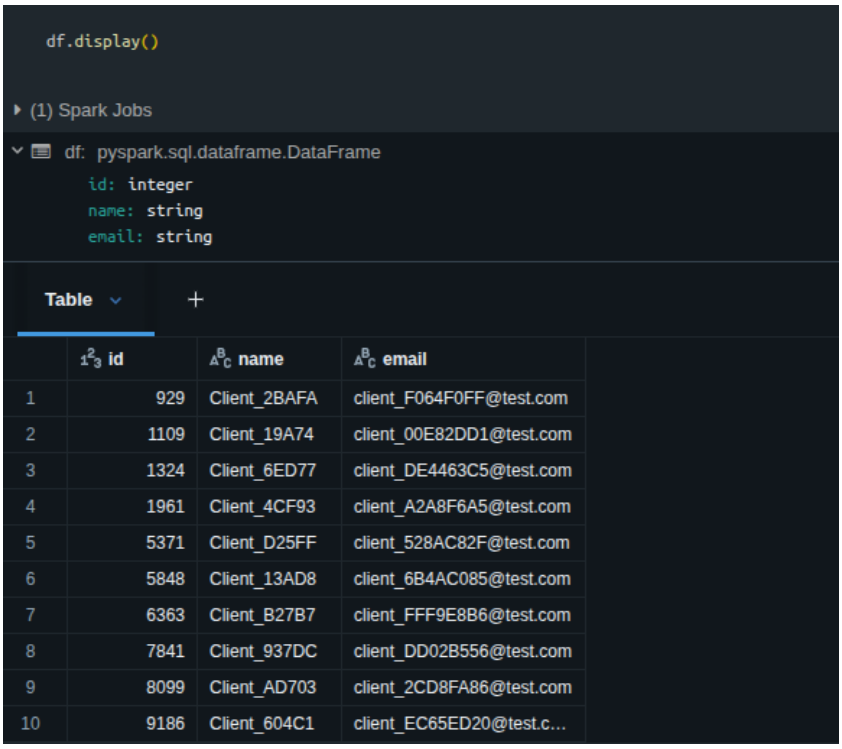


Figure 11: Notebook Databricks - alimentation table clients

Le fichier `sales.csv` est ingéré via un pipeline Azure Data Factory, outil natif conçu pour l'ingestion batch de fichiers plats sans transformation complexe.

Nom du pipeline	Début de l'exécution	Fin de l'exécution	Durée	Déclenchement effe...	État
pipeline_copy_sales_00_20	2/12/2026, 6:37:31 PM	2/12/2026, 6:37:56 PM	26s	Déclencheur manuel	Opération ...
pipeline_copy_clients_sql_10	2/12/2026, 6:36:57 PM	2/12/2026, 6:37:19 PM	23s	Déclencheur manuel	Opération ...
pipeline_copy_sales_00_20	2/12/2026, 6:36:35 PM	2/12/2026, 6:36:57 PM	23s	Déclencheur manuel	Échoué

Figure 12: Pipeline ADF - historique d'exécution

2. Intégration des métadonnées dans le catalogue

Les scans hebdomadaires Purview collectent automatiquement pour chaque source : - **Schéma** : colonnes, types de données - **Statistiques** : taille, nombre de lignes - **Classifications** : détection des données sensibles (emails, noms de personnes)

Asset name	Asset type
00-temp	Azure Data Lake Storage Gen2 File System
10-unprocessed	Azure Data Lake Storage Gen2 File System
20-raw	Azure Data Lake Storage Gen2 File System
50-entity	Azure Data Lake Storage Gen2 File System
90-out	Azure Data Lake Storage Gen2 File System
clients	Azure SQL Table
dbo	Azure SQL Schema
mezinebasenhood	Azure SQL Database
mezineserveursqlnhood2.database.windows.net	Azure SQL Server
mezinestorageaccnhood2	Azure Storage Account
mezinestorageaccnhood2	Azure Data Lake Storage Gen2 Service
tfstate	Azure Data Lake Storage Gen2 File System

Figure 13: Assets dans le Data Catalog

Overview | **Properties** | Schema | Lineage | Contacts | Related | Privacy

Filter by property key | **Show properties without a value**

Showing 7 of 12 properties

Properties

createTime	Friday, January 23, 2026 at 8:41 PM Central European Standard Time
modifiedTime	Friday, January 23, 2026 at 8:41 PM Central European Standard Time
name	clients
objectType	U
principalId	0
qualifiedName	mssql://mezineserveursqlnhood2.database.windows.net/mezinebasenhood/dbo/clients

Related assets

DbSchema	dbo

Figure 14: Propriétés de l'asset clients

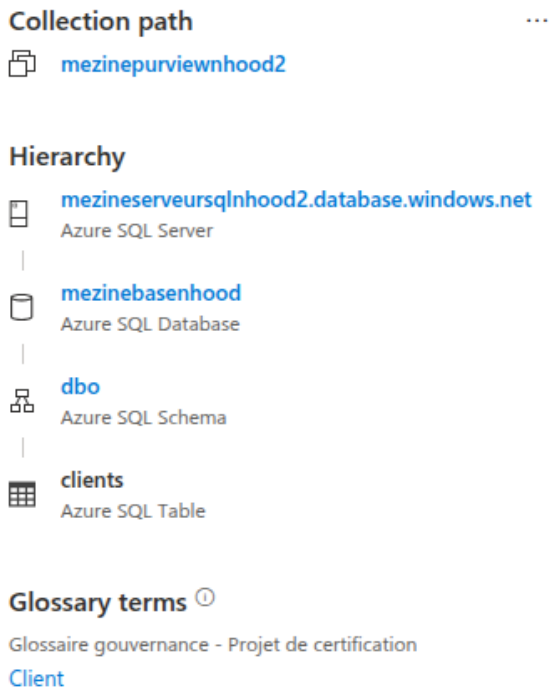


Figure 15: Hiérarchie et glossaire de l'asset clients

3. Données personnelles et RGPD

3.1 Registre des traitements

Traitement	Données concernées	Finalité	Base légale
Gestion clients	Nom, Email	Suivi de la relation client et communication	Exécution du contrat (Art. 6.1.b RGPD)
Analyse des ventes	Données de ventes (`sales.csv`)	Reporting et analyse de la performance commerciale	Intérêt légitime (Art. 6.1.f RGPD)

Durées de conservation : les données clients (nom, email) sont conservées 3 ans en base SQL avec purge automatique mensuelle, et jusqu'à 10 ans dans le Data Lake (couche 50-entity, lifecycle management policy). Les données de ventes sont conservées 5 ans conformément aux obligations comptables.

3.2 Procédure de tri des données personnelles

L'identification et le tri des données personnelles dans l'entrepôt reposent sur deux mécanismes :

Mécanisme	Type	Fréquence	Rôle
Classification automatique Purview	Automatisé	Hebdomadaire	Détecte et tagge les colonnes contenant des données sensibles
Glossaire métier Purview	Manuel	À chaque nouvelle source	Associe un terme métier aux colonnes pour documenter leur nature

Ce tri permet de distinguer les données personnelles (soumises au RGPD) des données non personnelles dans le catalogue.

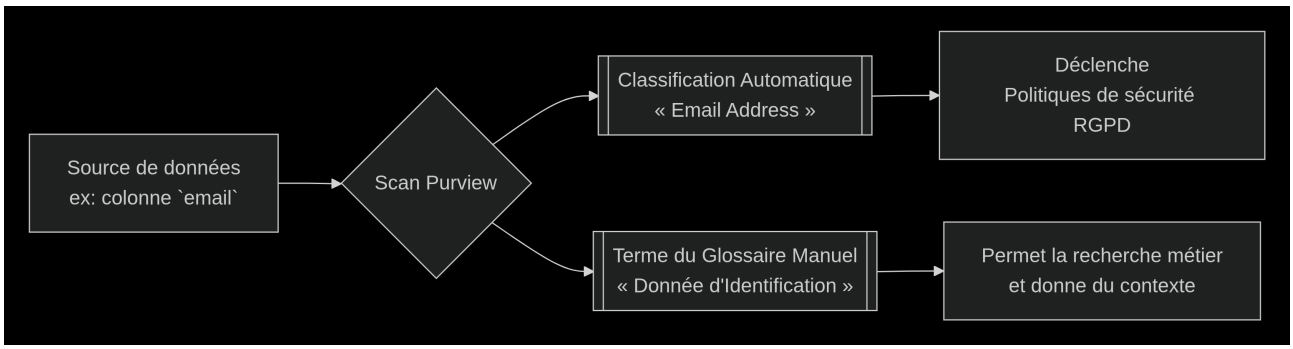


Figure 16: Schéma du processus de classification Purview

Figure 17: Termes du glossaire métier Purview

3.3 Données personnelles identifiées

La table `clients` contient les données personnelles suivantes :

Colonne	Type de donnée	Classification Purview	Terme glossaire
`name`	Nom de personne	Person Name	Donnée d'Identification
`email`	Adresse email	Email	Donnée d'Identification

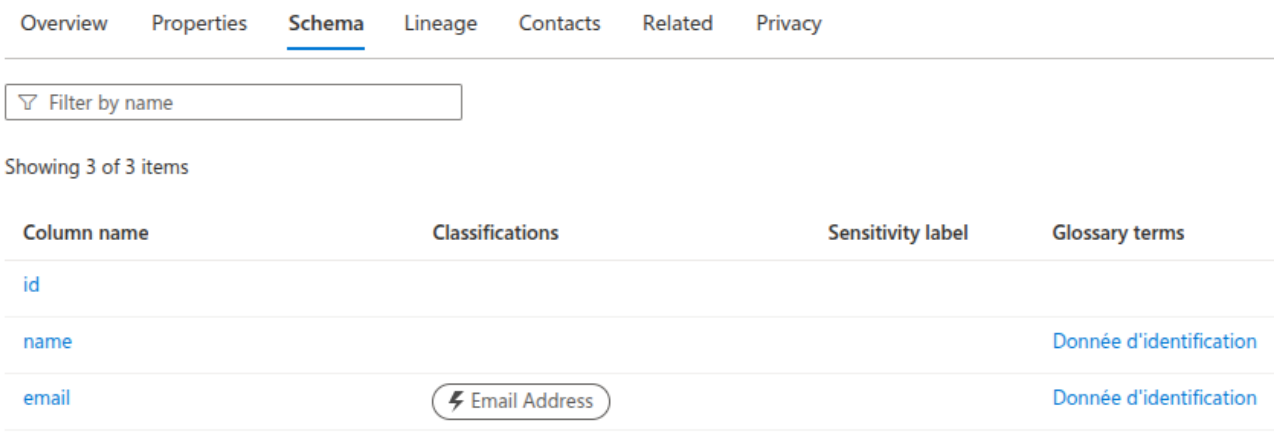


Figure 18: Classifications et termes glossaire sur la table clients

3.4 Procédures de conformité RGPD

Droit d'accès Procédure : Requête SQL pour extraire les données d'un client sur demande.

```
SELECT * FROM clients WHERE email = '<email_demandeur>';
```

Droit à l'effacement Procédure : Suppression des données personnelles sur demande.

```
DELETE FROM clients WHERE email = '<email_demandeur>';
```

> **Note** : Ces procédures doivent être formalisées dans un processus opérationnel avec validation, traçabilité et délais conformes (30 jours max).

Droit à la portabilité Procédure : Export des données au format CSV via Data Factory ou requête SQL.

4. Procédures de suppression

4.1 Politique de rétention du Data Lake (Lifecycle Management Policy)

La suppression automatique des fichiers du Data Lake est assurée par une **Azure Storage Lifecycle Management Policy**, déployée via Terraform (`azurerm_storage_management_policy`). Chaque couche dispose d'un délai de suppression adapté.

Couche	Container	Délai de suppression	Justification RGPD
Landing	`00-temp`	7 jours	Limitation stricte de la conservation
Brutes fiables	`10-unprocessed`	6 mois	Durée nécessaire pour la portabilité et le retraitement
Nettoyées	`20-raw`	2 ans	Alignement sur le cycle d'analyse métier
Transformées	`50-entity`	10 ans	Obligations légales de conservation (comptabilité)
Export	`90-out`	30 jours	Limitation dès que la finalité de l'export est remplie

Nom	État	Type d'objet blob
delete-temp-after-7-days	Activé	Bloquer
delete-unprocessed-after-180-days	Activé	Bloquer
delete-raw-after-730-days	Activé	Bloquer
delete-entity-after-3650-days	Activé	Bloquer
delete-out-after-30-days	Activé	Bloquer

Figure 19: Lifecycle Management Policy déployée

4.2 Purge des données personnelles en base SQL

Un notebook Databricks (`purge_clients_data`) supprime les clients dont la date de création dépasse 3 ans, conformément à la durée de conservation définie dans le registre des traitements. Un job Databricks planifie son exécution automatique le 1er de chaque mois.

```
> df_count: pyspark.sql.dataframe.DataFrame = [nb: integer]
Nombre de lignes à supprimer (clients > 3 ans) : 0
Suppression exécutée : 0 lignes supprimées.
```

Figure 20: Notebook purge_clients_data — exécution

5. Monitorage

Le monitorage est déployé via Terraform et repose sur **Azure Monitor**, **Log Analytics** (rétention 30 jours) et un **Action Group** qui notifie par email en cas d'alerte.

Chaque alerte s'évalue à une **fréquence** donnée sur une **fenêtre d'observation** : Azure agrège la métrique sur cette fenêtre et compare au seuil.

5.1 Alertes configurées

Composant	Métrique	Seuil	Fréquence	Fenêtre
Data Factory	`ActivityFailedRuns` (Total)	> 0	5 min	5 min
Storage Account	`UsedCapacity` (Moyenne)	$> 80\%$	1 h	6 h
SQL Database	`dtu_consumption_percent` (Moyenne)	$> 90\%$	5 min	15 min

L'alerte Data Factory déclenche dès le premier échec de pipeline, avec une fenêtre égale à la fréquence pour ne rien manquer sans compter en double. Pour le Storage Account, la capacité évoluant lentement, une fenêtre de 6 heures lisse les variations temporaires liées aux fichiers en transit. Pour la base SQL, une fenêtre de 15 minutes filtre les pics ponctuels tout en détectant rapidement une surcharge réelle.

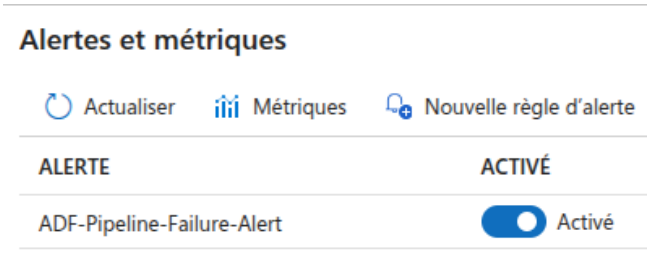


Figure 21: Alerte ADF Pipeline Failure activée

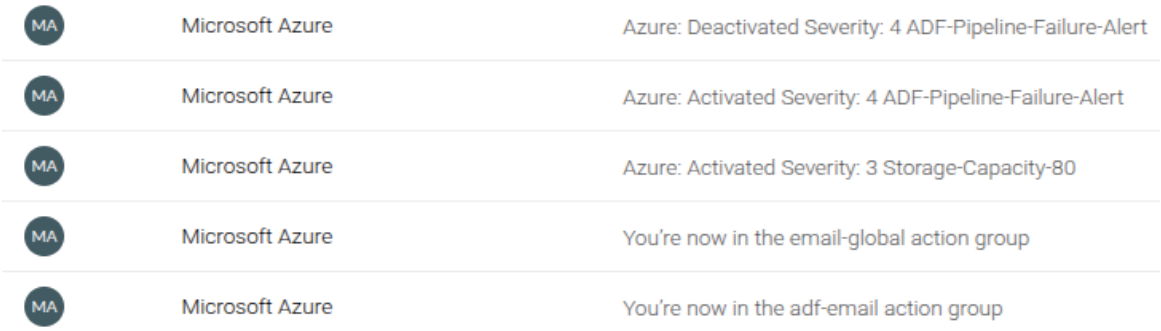


Figure 22: Emails d'alertes Azure reçus

5.2 Collecte des logs

Les logs `PipelineRuns` et `ActivityRuns` de Data Factory sont envoyés vers Log Analytics via un **Diagnostic Setting**, permettant l'investigation des échecs par requêtes KQL.

Nom	Type de ressource	Groupe de ressources	État des diagnostics
mezinefactorynhood2	Data Factory (V2)	mezinenhood	Activé
mezinestorageaccnhood2	Compte de stockage	mezinenhood	Désactivé

Figure 23: Diagnostic settings configurés

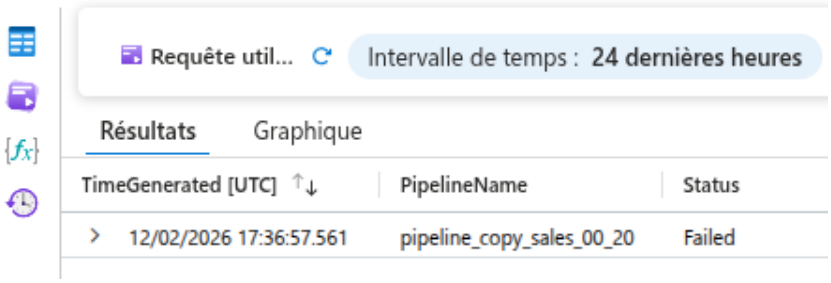
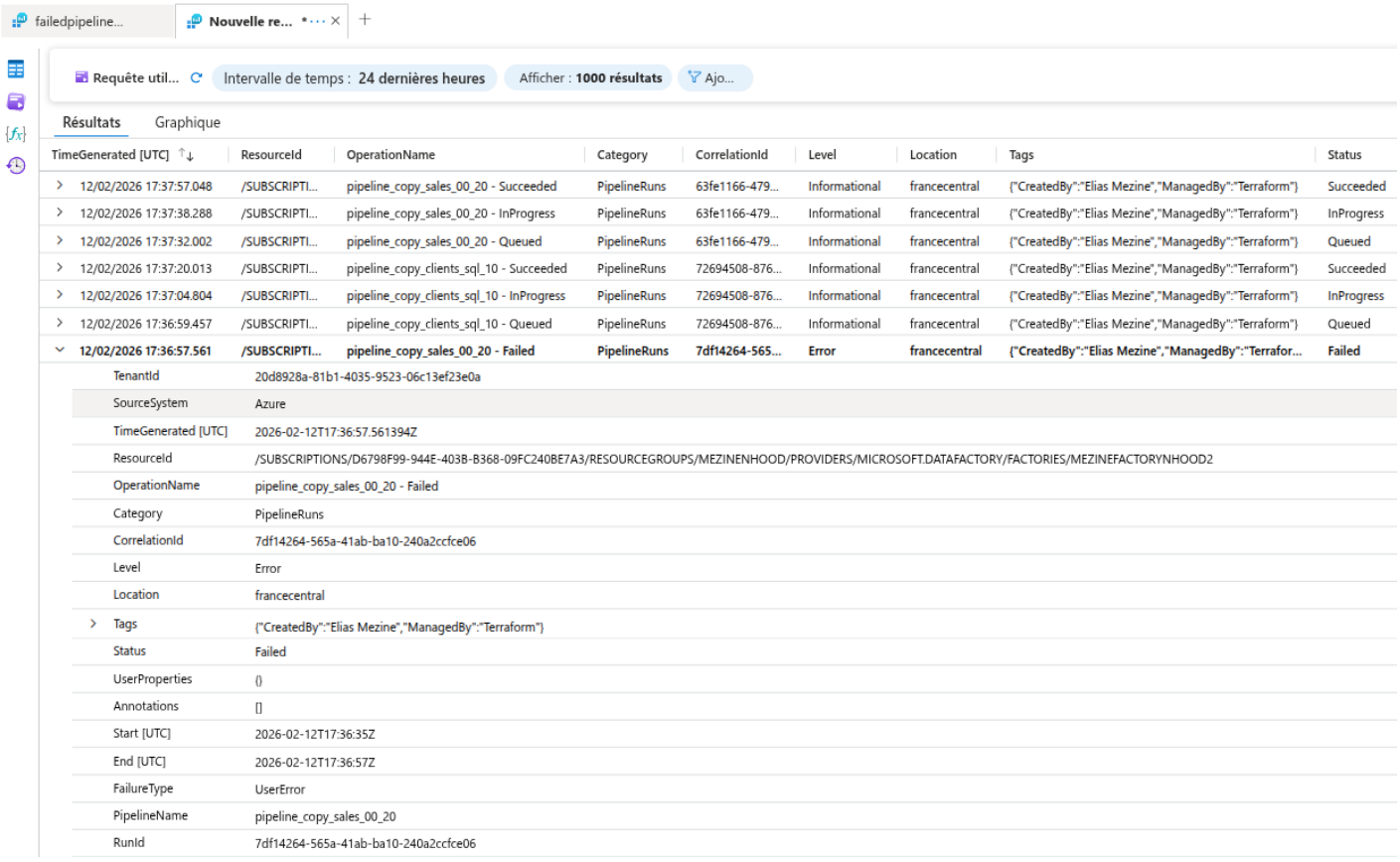


Figure 24: Log Analytics - pipeline échoué détecté



TimeGenerated [UTC]	ResourceId	OperationName	Category	CorrelationId	Level	Location	Tags	Status
12/02/2026 17:37:57.048	/SUBSCRIPTI...	pipeline_copy_sales_00_20 - Succeeded	PipelineRuns	63fe1166-479...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	Succeeded
12/02/2026 17:37:38.288	/SUBSCRIPTI...	pipeline_copy_sales_00_20 - InProgress	PipelineRuns	63fe1166-479...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	InProgress
12/02/2026 17:37:32.002	/SUBSCRIPTI...	pipeline_copy_sales_00_20 - Queued	PipelineRuns	63fe1166-479...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	Queued
12/02/2026 17:37:20.013	/SUBSCRIPTI...	pipeline_copy_clients_sql_10 - Succeeded	PipelineRuns	72694508-876...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	Succeeded
12/02/2026 17:37:04.804	/SUBSCRIPTI...	pipeline_copy_clients_sql_10 - InProgress	PipelineRuns	72694508-876...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	InProgress
12/02/2026 17:36:59.457	/SUBSCRIPTI...	pipeline_copy_clients_sql_10 - Queued	PipelineRuns	72694508-876...	Informational	francecentral	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}	Queued
12/02/2026 17:36:57.561	**/SUBSCRIPTI...**	**pipeline_copy_sales_00_20 - Failed**	**PipelineRuns**	**7df14264-565...**	**Error**	**francecentral**	**{"CreatedBy":"Elias Mezine","ManagedBy":"Terrafor...**	**Failed**

TenantId	20d8928a-81b1-4035-9523-06c13ef23e0a
SourceSystem	Azure
TimeGenerated [UTC]	2026-02-12T17:36:57.561394Z
ResourceId	/SUBSCRIPTIONS/D6798F99-944E-403B-B368-09FC240BE7A3/RESOURCEGROUPS/MEZINENHOOD/PROVIDERS/MICROSOFT.DATAFACTORY/FACTORIES/MEZINEFACTORYNHOOD2
OperationName	pipeline_copy_sales_00_20 - Failed
Category	PipelineRuns
CorrelationId	7df14264-565a-41ab-ba10-240a2ccfce06
Level	Error
Location	francecentral
Tags	{"CreatedBy":"Elias Mezine","ManagedBy":"Terraform"}
Status	Failed
UserProperties	{}
Annotations	[]
Start [UTC]	2026-02-12T17:36:35Z
End [UTC]	2026-02-12T17:36:57Z
FailureType	UserError
PipelineName	pipeline_copy_sales_00_20
RunId	7df14264-565a-41ab-ba10-240a2ccfce06

Figure 25: Log Analytics - détail des PipelineRuns

Implémentation des règles de gouvernance des données

1. Modèle de sécurité

1.1 Principes appliqués

La plateforme applique quatre principes de sécurité. Le moindre privilège garantit que chaque identité dispose uniquement des droits nécessaires à sa fonction. Les Managed Identities sont utilisées systématiquement pour éviter toute gestion de credentials. La séparation des rôles assure des droits distincts selon le service. Enfin, les droits sont attribués à des groupes Entra ID plutôt qu'à des individus.

2. Accès par groupes dans Azure Purview

Les droits d'accès à Azure Purview sont attribués à des **groupes** plutôt qu'à des individus, conformément au principe de gestion par groupes.

2.1 Création des groupes dans Microsoft Entra ID

Des groupes de sécurité sont créés dans Microsoft Entra ID pour regrouper les utilisateurs selon leur rôle dans la gouvernance des données.

Nom ↑	ID d'objet	Type de groupe
Groupe_AD_Purview_Admins	f577834d-7b91-4a77-9ad3-efb2c4db4d58	Sécurité
Groupe_AD_Purview_Consumers	f996a4aa-314f-413a-9483-f245da6c9668	Sécurité
Groupe_AD_Purview_Stewards	0054fe60-ef0c-4397-976b-54434b27aa23	Sécurité

Figure 26: Groupes de sécurité créés dans Microsoft Entra ID

2.2 Attribution des rôles Purview aux groupes

Chaque groupe se voit attribuer un rôle Purview adapté à ses besoins, en respectant le principe du moindre privilège.

Le groupe **Groupe_AD_Purview_Admins** dispose des rôles Collection Admin et Data Source Admin, lui permettant de gérer l'ensemble des sources et des collections du catalogue. Le groupe **Groupe_AD_Purview_Stewards** dispose du rôle Data Curator pour la curation et l'enrichissement du catalogue, notamment le glossaire et les classifications. Le groupe **Groupe_AD_Purview_Consumers** dispose du rôle Data Reader pour la consultation seule du catalogue.

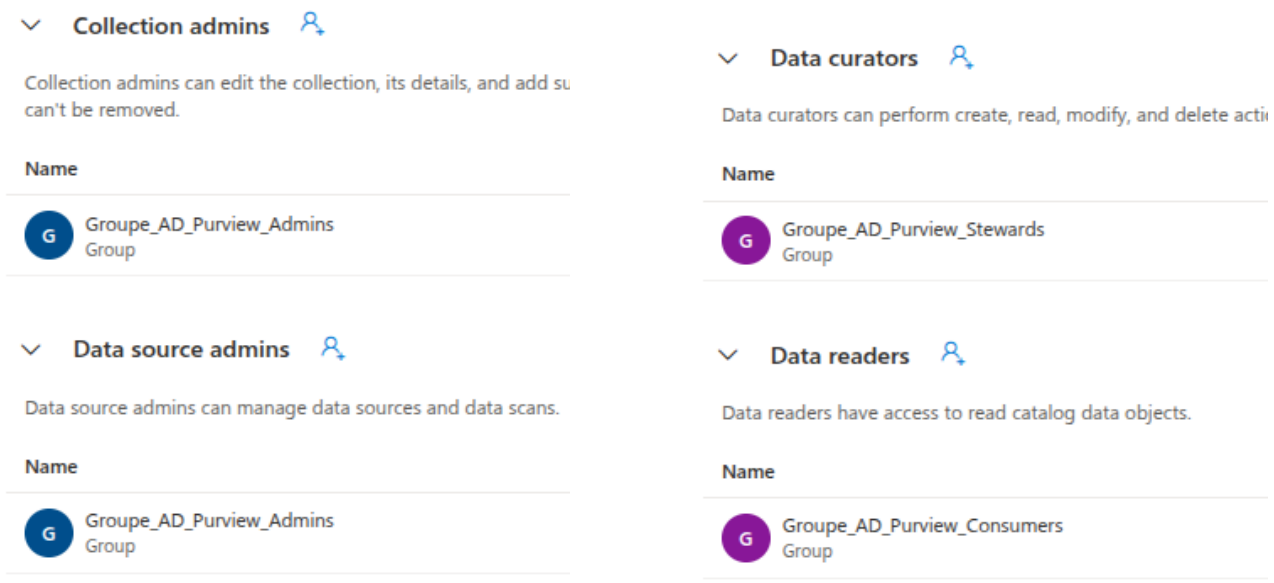


Figure 27: Attribution des rôles Purview aux groupes

2.3 Accès limités aux ressources nécessaires

Les groupes sont conçus pour que chaque rôle n'accède qu'aux fonctionnalités dont il a besoin :

- **Consumers** ne peuvent que consulter le catalogue — ils ne peuvent ni modifier les métadonnées, ni lancer de scans, ni gérer les sources
- **Stewards** peuvent enrichir le catalogue (glossaire, classifications manuelles) mais ne peuvent pas administrer les sources ni les collections
- **Admins** ont l'accès complet mais ce groupe est restreint aux administrateurs de la plateforme

3. Mécanismes d'authentification et droits des services

3.1 Comparatif des mécanismes d'authentification

Le projet utilise quatre mécanismes d'authentification selon les contraintes de chaque service.

L'identité managée système est créée et supprimée automatiquement avec la ressource : aucun credential n'est nécessaire, Entra ID gère tout. Elle est retenue pour les services Azure natifs ADF et Purview.

L'identité managée utilisateur présente un cycle de vie indépendant et est partageable entre ressources. Elle n'est pas retenue ici afin de maintenir une isolation stricte par service : si une identité partagée est compromise, tous les services qui l'utilisent sont exposés.

Le Service Principal requiert la gestion manuelle d'un secret stocké dans le Key Vault. Il est utilisé pour Databricks, service externe qui ne supporte pas les Managed Identities.

L'authentification SQL repose sur un mot de passe stocké dans le Key Vault et jamais exposé dans le code. Elle est inévitable pour les connexions JDBC à la base de données. Il est techniquement possible de mapper une Managed Identity sur un utilisateur SQL, mais cette approche n'est pas utilisée ici car Databricks ne supporte pas les MI.

3.2 Choix et justifications

Service	Mécanisme choisi	Justification
ADF	MI System	Service Azure, intégration native
Purview	MI System	Service Azure, intégration native
Databricks → Data Lake	Service Principal	Service externe, MI non supportée

Service	Mécanisme choisi	Justification
Databricks → Key Vault	SP `AzureDatabricks`	Nécessaire pour le Secret Scope
Databricks → SQL	Auth SQL	Connexion JDBC, MI non supportée
ADF → SQL	Auth SQL	Configuration SQL de la MI non gérée nativement par Terraform

Pourquoi pas l'identité managée utilisateur ? Chaque service dispose de sa propre identité avec des droits minimaux dédiés. Une MI utilisateur partagée entre services contredirait le principe de séparation des rôles : si un service est compromis, tous les services partageant cette identité seraient exposés. La MI système offre une meilleure isolation et un cycle de vie automatique.

Cas particulier de Databricks : Azure Databricks est un service externe (géré par Databricks Inc.) et ne supporte pas les Managed Identities pour accéder aux ressources Azure. Un Service Principal est nécessaire, avec son secret stocké dans le Key Vault et accessible via un Secret Scope.

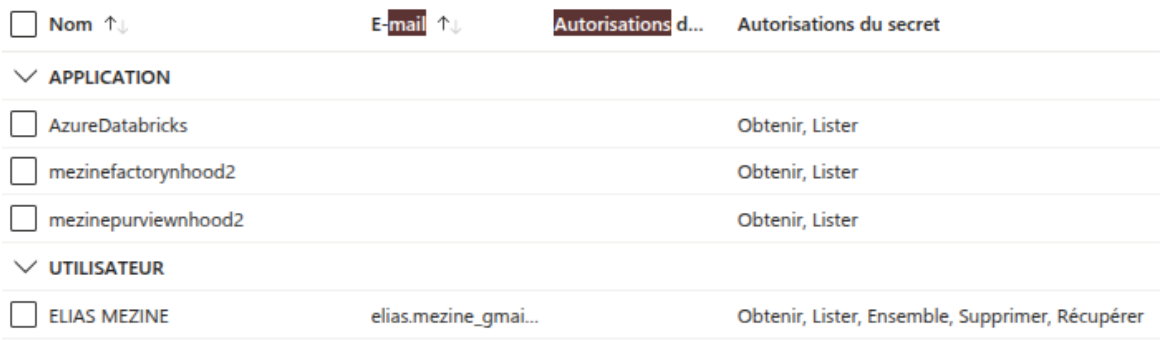
3.3 Matrice des droits par ressource

Data Lake (RBAC)

Principal	Type d'identité	Rôle
Utilisateur courant	Compte Azure AD	Storage Blob Data Contributor
Data Factory	MI System	Storage Blob Data Contributor
Purview	MI System	Storage Blob Data Reader
SP Databricks	Service Principal	Storage Blob Data Contributor

Key Vault (Access Policies)

Principal	Type d'identité	Permissions secrets
Utilisateur courant	Compte Azure AD	Get, List, Set, Delete, Recover
Data Factory	MI System	Get, List
Purview	MI System	Get, List
AzureDatabricks	SP auto-créé	Get, List



Nom	E-mail	Autorisations d...	Autorisations du secret
APPLICATION			
AzureDatabricks			Obtenir, Lister
mezinefactorynhood2			Obtenir, Lister
mezinepurviewnhood2			Obtenir, Lister
UTILISATEUR			
ELIAS MEZINE	elias.mezine_gmai...		Obtenir, Lister, Ensemble, Supprimer, Récupérer

Figure 28: Access Policies du Key Vault

SQL Database

Principal	Mécanisme	Accès
Utilisateur courant	Administrateur Azure AD	Admin SQL
Purview	MI System → RBAC Reader sur le serveur	Scan des métadonnées
Data Factory	Auth SQL (password depuis KV via linked service)	Lecture/écriture tables
Databricks	Auth SQL (password depuis KV via Secret Scope)	Lecture/écriture tables

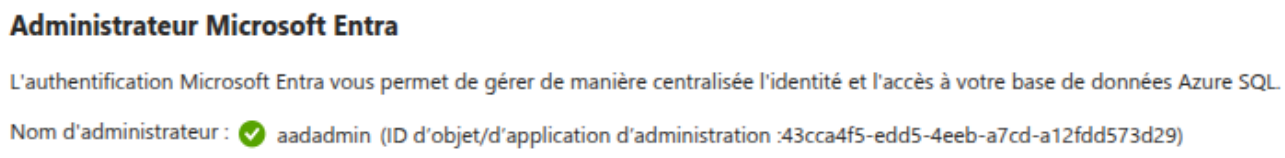


Figure 29: Administrateur Microsoft Entra du SQL Server

3.4 Contribution à la conformité RGPD

Le modèle d'accès mis en place contribue directement à la conformité RGPD :

- **Restriction des données personnelles** : l'accès en lecture seule des Consumers au catalogue empêche toute extraction non autorisée de données personnelles (`name`, `email`).
- **Traçabilité des accès** : les opérations d'authentification et les modifications de groupes ou de droits sont journalisées dans les journaux d'audit Microsoft Entra ID, permettant de retracer qui a obtenu ou perdu quels accès — condition nécessaire en cas de demande d'accès ou de contrôle réglementaire (CNIL).
- **Centralisation des credentials** : les mots de passe SQL sont stockés exclusivement dans le Key Vault, limitant l'exposition des accès aux données personnelles et offrant un point de contrôle unique pour les révocations.
- **Révocation immédiate** : le modèle par groupes permet de couper l'accès d'un utilisateur au catalogue Purview en le retirant de son groupe Entra ID, sans modifier les rôles Purview un par un.

4. Procédures de gestion des accès

1. L'utilisateur soumet une demande
2. Le manager valide le besoin métier
3. L'administrateur ajoute l'utilisateur au groupe Azure AD approprié
4. Révision périodique des accès : les propriétaires de groupes valident les accès actifs et suppriment ceux qui ne sont plus nécessaires
5. À chaque départ, suppression immédiate de l'utilisateur du groupe Azure AD

Synthèse

La plateforme de données terraNhood est construite sur Azure et déployée intégralement via Terraform, garantissant un déploiement reproductible et auditable. L'architecture s'organise autour d'un Data Lake Storage Gen2 en couches médaillon, Azure Data Factory pour l'orchestration batch et Azure Databricks pour le traitement programmatique. Le choix d'une architecture batch est délibéré : les fréquences journalières et hebdomadaires répondent pleinement aux besoins du projet, sans nécessiter de traitement temps réel.

L'infrastructure est intégralement opérationnelle : les 6 containers du Data Lake sont déployés, les 3 Linked Services ADF sont validés par des tests de connexion concluants, et Databricks accède au Data Lake via ABFS et à la base SQL via JDBC. Azure Purview catalogue les deux sources via des scans hebdomadaires qui détectent et classifient automatiquement les données personnelles. Le catalogue est alimenté par deux méthodes adaptées à chaque source : Databricks pour la base SQL et Azure Data Factory pour les fichiers CSV.

La conformité RGPD est assurée à deux niveaux : une politique de rétention différenciée par couche sur le Data Lake, et une purge mensuelle automatique des clients en base SQL au-delà de 3 ans. Le monitorage repose sur Azure Monitor avec des alertes sur les trois composants critiques, complété par Log Analytics pour l'investigation des incidents.

La gouvernance des accès applique le principe du moindre privilège : trois groupes Entra ID structurent les droits dans Purview selon les rôles, et chaque service technique dispose de sa propre identité managée système pour une isolation totale. Les credentials SQL inévitables — Databricks ne supportant pas les Managed Identities — sont centralisés dans le Key Vault et jamais exposés dans le code. Le modèle par groupes permet une révocation immédiate sans intervention sur les droits Purview individuels.